

CS 230 Outline Midterm Review 2

Polymorphism

OOP concept where a single interface can represent different underlying forms. In Java, it allows objects of different classes to be treated as objects of a common superclass.

Methods: + public, - private, # protected

- What is the use of **protected variables**?

Overriding vs. Overloading

Overriding (runtime polymorphism): Same signatures

Overloading (compile-time polymorphism): Same method different parameters

1. **Overriding** requires:
 - Same method name
 - Same parameter types **in the same order**
 - Appears in a subclass
2. **Overloading** requires:
 - Same method name
 - Different parameter types **or order**

Give an example Make sure **SIGNATURE!**

The Square Class

```
public class Square extends Rectangle
{
    /**
     * Constructor for objects of class Square
     */
    public Square(double width)
    {
        super(width, width);
    }
}
```

Super: When a child class overrides a method or hides a field, super allows access to the parent's version.

- super() must be the **first statement** in a child constructor (if used).

```
Public void makeSound() {
```

```

    super.makeSound(); // Calls Animal's makeSound()
    System.out.println(super.sound); // Accesses Animal's 'sound'
    System.out.println(sound); // Accesses Dog's 'sound' ("Bark")
}

```

Object Class

The Object class contains a few useful methods, which are inherited by all classes

I.e., the toString() method is defined in the Object class

Every time we define the toString method, we are actually **overriding** an inherited definition

Other examples you should be familiar with equals(), compareTo(), hashCode()

Abstract Classes vs interfaces:

Abstract:

- Classes that cannot be instantiated
- May contain abstract methods (without implementation) and concrete methods
- Represent an "is-a" relationship

Interfaces:

- Completely abstract types
- Define contracts that implementing classes must follow
- Represent "can-do" relationships
- Comparable Interface!

Similarities

- Both cannot be instantiated directly
- Both are used to achieve abstraction
- Both can be used to achieve polymorphism
- Both can contain abstract methods

Use Abstract Classes When: (Abstract **extends**)

- You want to share code among closely related classes
- You need to define a template with some implemented methods
- You need a constructor

Use Interfaces When: (Interface **Implements**)

- You want to define a contract for unrelated classes
- You need multiple inheritance of type
- You want to specify what a class can do (behavior)

Linked Lists

What are Java Generics?

Generics enable types (classes and interfaces) to be parameters when defining classes, interfaces, and methods. They allow for type safety and eliminate the need for explicit type casting.

In Java, T is often used as a generic type parameter, most commonly seen in generic classes, methods, or interfaces. It stands for "Type", and it's a placeholder for any object type.

Importance of Generics

Without generics we would:

- Need to use Object types and explicit casting everywhere
- Have less readable code (full of type casts)
- Be unable to create type-safe collections

LinkedList vs Arrays

Benefits of LinkedLists - take advantage when working with one side of the list

- **Dynamic size:** Grows/shrinks as needed (no fixed capacity)
- **Efficient insertions/deletions:** $O(1)$ at head, $O(n)$ elsewhere (no shifting needed)

Benefits of Arrays

- **Random access:** $O(1)$ access to any element
- **Less overhead:** No need for node objects/pointers

Time Complexity Comparison

Operation	Array	Linked List
Access	$O(1)$	$O(n)$ or $O(1)$ for first node
Insert at head	$O(n)$	$O(1)$
Insert at tail	$O(1)^*$	$O(n)$
Delete at head	$O(n)$	$O(1)$
Search	$O(n)$	$O(n)$

Linked Lists work great for things that operate on the front of a list.

Stacks

First in Last Out

Queues

First in first out

Sorting Algorithms

Selection Sort:

SelectionSort orders values by repeatedly putting a particular value into its final sorted position

The algorithm:

1. find the smallest value in the dataset
2. switch it with the value in the first position
3. find the next smallest value in the dataset
4. switch it with the value in the second position
5. repeat until all values are in their sorted positions

Insertion Sort:

InsertionSort orders a list of values by repeatedly inserting a particular value into a sorted subset of the list

The algorithm:

1. consider the first item to be a sorted sublist of length 1
2. insert the second item in the sorted sublist,
3. shifting the first item if needed
4. insert the third item into the sorted sublist,
5. shifting the other items as needed
6. repeat until all values inserted into their proper positions

Merge sort:

Merge sort orders a list of values by recursively dividing the list in half until each sublist has one element, then recombining (merging) the sublists

The algorithm:

Split: Divide the list into two equal parts—left and right. If the list has odd length, one half will have an extra element.

Sort: Recursively use merge-sort to sort the two lists. Note that by definition, a list of one element is sorted (base case).

Merge: Merge the two sorted parts into one sorted list

Algorithm	Best	Average	Worst	Space in addition to original array
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$

Binary Search

How does it work?

1. Start with two pointers: `low` and `high`.
2. Find the middle: `mid = (low + high)/2`.
3. If `target == arr[mid]`, return index.
4. If `target < arr[mid]`, search left half.
5. If `target > arr[mid]`, search right half.
6. Repeat until found or interval is empty.

Runtime complexity:

- Time: $O(\log n)$

HASH TABLES

- **What is a hash table?**
A data structure that maps keys to values using a **hash function**.
- **Use cases:**
Fast lookups, insertions, deletions (e.g., dictionaries, caches).
- **Hash function:**
Converts a key to an index. Good hash functions minimize collisions.
- **Good vs. Bad:**
 - *Good*: Uniform distribution.
 - *Bad*: Clusters values, many collisions.
- **Complexities:**
 - Average Time: $O(1)$ for lookup/insert/delete.

- Worst-case: **$O(n)$** (due to collisions).
 - Space: **$O(n)$**
- **Collision resolution:**
 - *Separate chaining*: Each index has a linked list of entries.
 - *Linear probing*: Find next open slot.
- **Load factor:**

Ratio of number of entries to table size.

 - Higher load $\rightarrow$ slower, more collisions.
 - Resize table when load > threshold (e.g., 0.75).

Time and Space Complexity

- What is Big-O? - in reference to unknown variables - inputs, eg size of an array
- Why do we omit constants? - think if n was arbitrarily large

TREES

Definitions & Applications

- What's a tree?

A connected acyclic graph with a **single root node**.
- Similarities/Differences with Graphs:
 - Tree is a special graph (no cycles, one root).
 - Graphs can have cycles and multiple roots.

Preorder - Visit node, traverse left, traverse right

Inorder - left, visit, right

Postorder - left, right, visit

Level-order - uses queue, visits all children first, similar to **BFS**